CAPÍTULO XI

Recomendaciones

XI.1. Infraestructura complementaria: un pipeline de MLOps para reentrenamiento continuo

El capítulo de Alcance excluye explícitamente el aprendizaje continuo y el reentrenamiento automático embebido en el proceso de la aplicación anfitriona, por ser incompatible con el principio de operación autónoma sin dependencias externas que rige el diseño de logSguarDian en tiempo de ejecución. Esa exclusión sigue vigente: la librería npm entregada no reentrena modelos dentro del proceso de la aplicación que protege. Durante el desarrollo del proyecto, sin embargo, el equipo construyó y verificó, como infraestructura separada y exploratoria –no como parte del paquete npm ni de sus dependencias en tiempo de ejecución–, un pipeline de ocho fases para el ciclo completo de recolección de telemetría, reentrenamiento y despliegue controlado de nuevas versiones del modelo híbrido. Se documenta aquí como recomendación de arquitectura de producción para quien adopte logSguarDian a escala, y no como parte del alcance evaluado en los objetivos específicos de este trabajo.

La motivación de esta exploración es directa: cada cambio al pipeline de extracción de características exige un reentrenamiento completo del modelo, condición que el proyecto siguió de forma estricta a lo largo de sus más de diez ciclos de reentrenamiento documentados. Un despliegue real de logSguarDian a múltiples aplicaciones cliente enfrentaría el mismo problema de forma recurrente, y sin un mecanismo que sistematice la recolección de evidencia de producción, la curación de esa evidencia y la validación de un modelo candidato antes de promoverlo, cada ciclo de mejora tendría que repetirse manualmente. El pipeline construido –bajo el directorio `packages/mlops` del repositorio del proyecto– explora si ese proceso puede automatizarse sin comprometer la garantía de bloqueo del modelo en producción.

Las ocho fases del pipeline son las siguientes:

1. **Recolección de telemetría.** Un endpoint dedicado recibe únicamente el vector de 76 características de cada petición clasificada, nunca el payload crudo, con el mismo patrón de notificación asíncrona sin bloqueo (*fire-and-forget*) que ya emplea el sistema de webhooks de la librería.

2. **Simulación de flota.** Múltiples fuentes simuladas envían telemetría de forma concurrente para poblar un corpus de validación representativo de un despliegue con varios clientes.

3. **Agrupamiento de anomalías.** El componente no supervisado (Isolation Forest) y un algoritmo de agrupamiento por densidad (DBSCAN) se aplican sobre el espacio de 63 características que consume el modelo no supervisado, para aislar patrones estadísticamente atípicos sin necesidad de una etiqueta previa.

4. **Curación humana.** Una interfaz de línea de comandos presenta los grupos resultantes para revisión manual y genera un archivo con el mismo esquema que consume el script de unificación del corpus de entrenamiento del proyecto.

5. **Reentrenamiento orquestado.** Un script coordina de punta a punta la unificación del corpus curado, el reentrenamiento del modelo híbrido y la generación de los artefactos ONNX correspondientes, y reutiliza el mismo pipeline de entrenamiento documentado en el capítulo de Alcance.

6. **Validación automática mediante cinco criterios de aceptación (*gates*).** Se verifican cinco criterios, en orden creciente de costo computacional y con interrupción inmediata ante el primer fallo (*fail-fast*). Son el contrato de features del modelo candidato, sus métricas de precisión y recall frente al conjunto de prueba bloqueado, la paridad de predicción entre el modelo en Python y su versión serializada en ONNX, su desempeño sobre la suite de pruebas end-to-end, y su huella de memoria bajo la arquitectura real de *worker pool* de producción.

7. **Despliegue canario.** Un modelo candidato que aprueba los cinco criterios anteriores se evalúa mediante dos mecanismos: reemplazo del corpus de prueba etiquetado como mecanismo primario de decisión, y una ventana acotada de sombra sobre tráfico en vivo como mecanismo secundario, nunca como proceso continuo indefinido.

8. **Promoción o reversión.** La promoción de un modelo candidato a producción exige el cumplimiento simultáneo de un criterio estricto –cero casos en los que el candidato deje pasar un ataque que el modelo en producción bloquea correctamente– y un criterio de concordancia general de al menos 99.5 % entre los veredictos de bloqueo del candidato y los del modelo vigente sobre el corpus de reemplazo.

Como prueba de concepto, el pipeline de reentrenamiento se ejecutó el 29 de agosto de 2026 sobre telemetría simulada de 244 eventos. Cinco fuentes simuladas (sim-host-1 a sim-host-5) aportaron 238 eventos, que reparten el corpus de payloads de prueba. Una fuente dedicada (sim-host-novel) aportó 6 eventos: seis réplicas de un payload poliglota diseñado para combinar, en una sola petición, sintaxis de las cuatro categorías de ataque cubiertas.

El reporte de agrupamiento (DBSCAN con `eps` 2.5 y `min_samples` 3) registra 183 eventos anómalos y 16 grupos, además de un grupo de ruido de 53 eventos. Las seis réplicas formaron un grupo propio, con puntaje de anomalía promedio de -0.153, inferior al umbral de 0.002486 que usa el script de agrupamiento (el de `if_v9`), y el componente supervisado las clasificó como XSS. Según un comentario del script generador, una versión anterior del script, con un payload ofuscado de una sola clase, no produjo esa separación; esa versión no se conserva en el repositorio. Dos cautelas acotan el resultado: las seis peticiones son réplicas idénticas, por lo que su agrupación no mide la capacidad de generalizar a variantes distintas, y el reporte proviene de una sola ejecución local, conservada fuera del repositorio (SHA-256 `7fb882d5...e7e6`).

Se recomienda a quien adopte logSguarDian en un despliegue con varias aplicaciones cliente convertir esta prueba de concepto en un servicio verificado antes de usarla para actualizar modelos en producción. La acción comprende repetir la ejecución con telemetría de tráfico real, no simulada, y con variantes distintas entre sí, y comprobar con ella los cinco criterios de aceptación de la fase 6. Se espera determinar si el agrupamiento aísla variantes no vistas y si el reentrenamiento orquestado produce un candidato que cumpla los criterios de promoción. Se recomienda mantener la separación entre la librería embebida y la infraestructura externa que sostiene su mejora continua.

XI.2. Ampliación del corpus benigno

Se recomienda ampliar el corpus benigno con tráfico representativo de cada aplicación objetivo y volver a medir los falsos positivos en aplicaciones de terceros con el mismo protocolo descrito en la subsección VIII.3.7.

Esa ampliación debe incluir tráfico de clientes HTTP diversos y cadenas de *User-Agent* variadas, dado el peso de esa característica en el modelo y el nivel de bloqueo observado en aplicaciones de terceros (sección VIII.3.7). Se espera una tasa de bloqueo de tráfico benigno menor que la de la subsección VIII.3.7, medida con el mismo protocolo.

XI.3. Validación de la versión publicada

La versión 0.1.0 del paquete contiene `rf_v10` e `if_v9`, mientras que las métricas de modelo de este trabajo corresponden a `rf_v11` e `if_v10`. El 11 de octubre de 2026 se publicó la versión 0.1.1, que contiene esos modelos y el cambio del commit `25c7e64` en la forma en que el middleware entrega la query al extractor (anexo XIII.6). El coautor ejecutó la Configuración 2 sobre la Ronda 5 con esa versión instalada desde el registro (anexo XIII.8). No consta que se haya repetido con ella el experimento de latencia, memoria y CPU.

Se recomienda repetir ese experimento en Linux nativo con la versión 0.1.1 del registro, e identificar el paquete por su SHA-256 en las Configuraciones 1, 3a y 3b sobre la Ronda 5. Se recomienda también medir el desajuste entre la cadena de consulta del entrenamiento y la que recibe el middleware, que este trabajo no midió. Se espera saber si los criterios de Δp95, memoria y CPU se cumplen con el paquete que se distribuye y no con un tarball de validación. Hasta entonces, las mediciones de integración de este trabajo con `rf_v11` e `if_v10` corresponden a tarballs de validación, salvo las dos corridas de la Configuración 2 del coautor, y las cifras de latencia, memoria y CPU corresponden al par B.

XI.4. Costo de memoria y CPU por componente

Los incrementos de memoria (de 331.9 a 375.9 MiB) y de CPU (de +529.6 % a +614.4 % en carga normal y de ataque) superan los criterios del protocolo, y este trabajo no los atribuyó a ningún componente. Se recomienda medirlos en el mismo entorno para tres configuraciones: solo Random Forest, Random Forest con un *worker* de Isolation Forest, y la configuración actual de tres `worker_threads`. Con esas mediciones debería evaluarse también `IF_POOL_SIZE` igual a 1. El resultado de esas mediciones no se establece aquí.

Se espera atribuir el incremento a uno o varios componentes y determinar si alguna de esas configuraciones acerca el consumo a los criterios del protocolo. Se recomienda repetir con ellas el

escenario de volumen y registrar los tiempos de espera agotados (subsección VIII.3.2), porque con la configuración actual el 7.00 % de las peticiones los agotó.

XI.5. Corpus de evaluación fuera del entrenamiento

El corpus de `xss` se construyó después de conocer los resultados de la Ronda 5, y un caso hallado con ZAP no basta para confirmar su explotabilidad. El coautor ejecutó las Configuraciones 1, 3a y 3b el 25 de octubre de 2026 (anexo XIII.8). Se recomienda construir un corpus de `xss` independiente de los resultados y repetir la comparación con el cortafuegos de aplicaciones web sobre él, con confirmación de la explotabilidad en más de un caso. Se recomienda también definir para cada payload su forma exacta en el cable, para que el arnés no la altere. Con el tarball del par A, el límite inferior del intervalo de Wilson de `path_traversal` (79.4 %) queda bajo el criterio del 80 %; la medición debe repetirse con un número mayor de payloads.

Se recomienda también construir un conjunto de variantes de ataque ausentes del corpus de entrenamiento y medir con él el recall de Isolation Forest y el de Random Forest. El objetivo general incluye la detección de variantes desconocidas, y hoy solo tiene evidencia indirecta (capítulo X). Se espera obtener evidencia directa sobre esa detección y decidir, con una incertidumbre menor, si la cobertura de `xss` y la de `path_traversal` con el par A cumplen el criterio de 80 %.

XI.6. Rigor del protocolo de evaluación

Las particiones de validación y prueba comparten la estructura de casi-duplicados del entrenamiento, y la exclusión por fuente muestra una dependencia marcada de la fuente en varios casos (anexos XIII.2 y XIII.5). Se recomienda construir las particiones con agrupamiento de casi-duplicados y reservar fuentes completas para validación y prueba. El conjunto de prueba se leyó tres veces para la generación vigente, y su lock se re-fijó después del reentrenamiento (anexo XIII.3). Se recomienda registrar cada lectura y escribir el lock en un commit previo al entrenamiento. Se espera que las métricas de prueba midan la generalización entre fuentes y que cada cifra tenga una lectura única verificable.

Las métricas de prueba se calcularon por argmax, sin la regla de decisión desplegada, y la validación actual discrimina poco entre los umbrales 0.20 y 0.40 (anexo XIII.1). Se recomienda calcular las métricas de prueba con la regla desplegada y revalidar el umbral de 0.35 sobre una partición con menos casi-duplicados. Se recomienda evaluar Isolation Forest fuera de validación, porque su umbral se eligió sobre ese mismo conjunto. Se recomienda también versionar el generador de las filas sintéticas de `cmdi` y revisar el etiquetado de las técnicas de tiempo ciego, que el modelo clasifica como `sqli` (sección VIII.5). Se espera decidir si el umbral y las métricas actuales resisten una partición independiente.

Vo.Bo.: (f) _____
Ing. Erick Francisco Marroquín Rodríguez

Fecha de aprobación: Guatemala, _____ de _____ de 2026.

Versión del 13 de octubre de 2026